Parul® University
Vadodara, Gujarat

NAAC GRADE A++

Information and Communication Technology

Arrays

Study Guide

Mr. Shivkumar Lilhare
Assistant Professor(CSE), PIT
Parul University

4.1.1 Arrays

An array in Java is a data structure that allows you to store multiple values of the same type in a single variable instead of declaring separate variables for each value.

◆ Characteristics of Arrays:

- **Fixed Size**: Once created, the size of the array cannot change.
- **Homogeneous Data:** All elements must be of the same data type.
- **Indexed:** Array elements are indexed (first element is at index 0).
- **Contiguous Memory:** Stored in continuous memory blocks for fast access.

◆ Visual Representation:

```
int[] arr = {10, 20, 30, 40};
Index:    0  1  2  3
Value:    10 20 30 40
```

◆ Why Arrays Are Needed in Java

◆ Without Arrays:

To store 100 student marks:

```
int s1, s2, s3, ..., s100; // Tedious and inefficient
```

◆ With Array:

```
int[] marks = new int[100]; // Compact and manageable
```

◆ Benefits:

- Easier to manage large data.
- Supports iteration using loops.
- Efficient memory allocation.
- Used as the base for advanced structures like lists, stacks, and queues.

◆ Array Declaration in Java

- **Syntax:**

```
datatype[] arrayName;
```

or

```
datatype arrayName[];
```

Array Initialization

There are two types:

A. Static Initialization

- Declaration + Initialization in one step:
  int[] arr = {10, 20, 30, 40, 50};

B. Dynamic Initialization

- Declare the array and assign values later:
  int[] arr = new int[5]; // Creates array of size 5
  arr[0] = 10;
  arr[1] = 20;
  // Remaining values default to 0

4.1.2 Indexing and Accessing Elements

What is Indexing?

- Each element in an array is identified by an **index**, starting from 0.
- The last index is array.length - 1.

Accessing Elements:

Syntax:

```
arrayName[index];
```

Example:

```
int[] numbers = {10, 20, 30, 40};
System.out.println(numbers[0]); // Output: 10
System.out.println(numbers[3]); // Output: 40
```

Modifying Elements:

```
numbers[2] = 100; // Replaces 30 with 100
```

⚠ IndexOutOfBoundsException

- Trying to access an index outside the array size throws an exception:
  System.out.println(scores[3]); // Error!

The length Property

- Each array has a public final field named .length, which tells how many elements the array can hold.

Example:

```
int[] nums = {2, 4, 6, 8};
System.out.println(nums.length); // Output: 4
```

- .length is **not a method**, it is a **variable**:

```
// Correct
arr.length

// Incorrect
arr.length() // ✕
```

Internal Memory Layout of Arrays in Java

Behind the Scenes:

- **Arrays are objects in Java.**
- **Stored in heap memory.**
- **Each element is stored in contiguous memory blocks.**
- **Java maintains:**
  - **A reference to the array object**
  - **A header with the array length and type**
  - **The actual element values in order**

Example:

```
int[] arr = new int[3]; // allocates memory for 3 int elements
```

Memory Diagram:

```
arr → [0] [0] [0]
       0   1   2
```

Default Values of Array Elements

When arrays are created in Java, elements are initialized to **default values** based on their data type:

Data Type	Default Value
byte, short, int, long	0
float, double	0.0
char	'\u0000' (null char)
boolean	false
Object references	null

🔶 **Example:**

```
int[] a = new int[3]; // [0, 0, 0]
boolean[] flags = new boolean[2]; // [false, false]
String[] names = new String[2]; // [null, null]
```

➢ Summary of Key Concepts:

- **Definition: An array is a collection of fixed-size elements of the same data type stored in contiguous memory locations.**
- **Need: Useful for storing and managing large sets of data efficiently.**
- **Declaration & Initialization:**
- **Syntax: int[] arr = new int[5]; or int[] arr = {1, 2, 3};**
- **Usage: Accessed via indices starting from 0.**
- Indexing: Arrays use zero-based indexing.
- Accessing Elements: Via arr[index]; throws exception if index is out of range.
- Length Property: arr.length returns number of elements.
- Memory Layout: Arrays are objects stored in heap memory with continuous elements.
- Default Values: Elements are automatically initialized based on type (0, false, null, etc.).

➢References:

- **Oracle Java Docs – Arrays:**

  https://docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html

- **Geeks for Geeks – Arrays in Java:**

  https://www.geeksforgeeks.org/arrays-in-java/

- **JavaTPoint – Array in Java:**

  https://www.javatpoint.com/array-in-java

- **W3Schools – Java Arrays:**

  https://www.w3schools.com/java/java_arrays.asp

- **"Java: The Complete Reference" by Herbert Schildt**

Parul® University
Vadodara, Gujarat

NAAC GRADE A++

https://paruluniversity.ac.in/